

Cuaderno de Bitácora

TFG DAM — Stockly

Adrián Bravo Santos · Miguel Ángel Florido
Generado el 2026-05-16



Cuaderno de bitácora — TFG DAM "Stockly"

Trabajo Fin de Grado · Ciclo Formativo de Grado Superior — DAM Autores: **Adrián Bravo Santos** y **Miguel Ángel Florido** Tutor/a: *por completar* Curso académico: 2025-2026

Este documento registra el día a día del proyecto: decisiones, problemas encontrados, soluciones aplicadas, reparto de trabajo y reflexión técnica. El desarrollo se ha llevado a cabo a lo largo de **dos semanas** (Día 1 a Día 14), trabajando de forma intensiva por iteraciones cortas.

0. Datos del proyecto

Campo	Valor
Título	Stockly — Gestión inteligente de almacén y reservas
Tipo	Aplicación web multiplataforma (PWA, escritorio + móvil)
Stack	Node.js + Express · MariaDB · HTML/CSS/JS (PWA)
Repositorio	https://github.com/Husslesnake/TFGDAM
Entorno desarrollo	Windows 11 + VS Code · Node 18 LTS · MariaDB 10.11
Metodología	Iterativa por sprints cortos (1-4 días) con tablero GitHub Projects
Duración total	2 semanas (Día 1 → Día 14)

Reparto general de roles

Área	Responsable principal	Apoyo
Modelado de datos y base de datos	Miguel Á. Florido	A. Bravo
API REST (rutas, autenticación, lógica)	A. Bravo Santos	Miguel Á.
Frontend (UI/UX, vistas, PWA)	A. Bravo Santos	Miguel Á.
Panel admin / dashboard / reporting	Miguel Á. Florido	A. Bravo
Despliegue, dominio, hosting	A. Bravo Santos	Miguel Á.
Documentación + memoria	Miguel Á. Florido	A. Bravo

El reparto es orientativo: ambos hacen revisión cruzada de Pull Requests.

1. Decisiones técnicas clave

Día	Decisión	Motivo
Día 1	Stack Node + Express + MariaDB + vainilla JS	Cubre todas las competencias del módulo de Acceso a Datos y Programación Multimedia sin dependencias frágiles
Día 2	PWA en lugar de app nativa para la versión móvil	Mismo código sirve a escritorio y móvil; se puede empaquetar con Capacitor si se requiere APK
Día 3	JWT + bcrypt para autenticación	Estándar industrial, libre de sesiones de servidor, fácil de explicar en defensa
Día 4	Reservas con <code>SELECT ... FOR UPDATE</code>	Garantiza que dos usuarios no reserven el último ítem a la vez
Día 7	Tabla <code>movimientos</code> para auditoría	Demuestra trazabilidad real, requisito importante en almacenes
Día 9	Modo oscuro y bottom navigation	Ergonomía móvil + accesibilidad
Día 10	Dashboard con gráficos en CSS puro	Evita dependencia pesada de Chart.js para una visualización sencilla

2. Sprints / iteraciones

Sprint 0 — Análisis y arranque (Día 1 → Día 2)

- ✓ Definición del problema: gestión de reservas en almacén con concurrencia.
- ✓ Identificación de casos de uso por rol.
- ✓ Diagrama E/R inicial (usuarios, categorías, productos, reservas).
- ✓ Repositorio Git creado, README inicial.
- ✓ Esquema SQL con 500 productos semilla.
- **Aprendizajes:** importancia de definir la cardinalidad antes de tocar código.

Sprint 1 — Prototipo funcional (Día 3 → Día 4)

- ✓ API REST básica (productos, reservas) sin auth.
- ✓ Frontend HTML/JS con grid de productos y modal de reserva.
- ✓ Reservas con bloqueo transaccional (problema corregido tras detectar carrera en pruebas).
- 🐛 Bug: el contador `stock_reservado` quedaba descuadrado al cancelar reservas concurrentes → resuelto envolviendo en transacción.
- **Aprendizaje:** cualquier cambio que afecte a stock debe ir dentro de una transacción.

Sprint 2 — Autenticación y roles (Día 5 → Día 6)

- ✓ Login + registro + JWT.
- ✓ Middleware `authRequired` y `requireRole`.
- ✓ Rate-limit en `/api/auth`.
- ✓ Roles cliente / operario / admin.
- ⚠ Decisión: la BD guarda únicamente el hash; las contraseñas semilla se generan en el primer arranque del backend.

Sprint 3 — Panel admin y dashboard (Día 7 → Día 8)

- ✓ Endpoint `/api/admin/stats`.
- ✓ KPIs, gráficos en CSS y top productos.
- ✓ Tabla de movimientos con auditoría.
- ✓ Export CSV de reservas.
- **Aprendizaje:** agregar columnas `creado_en`, `actualizado_en`, `activo` desde el principio nos hubiera ahorrado migraciones.

Sprint 4 — UI/UX y PWA (Día 9 → Día 10)

- ✓ Sistema de diseño con tokens (modo claro y oscuro).
- ✓ Bottom navigation y diseño responsive.
- ✓ Manifest + service worker.
- ✓ Iconos generados desde un único SVG.

Sprint 5 — Despliegue, instalador y documentación (Día 11 → Día 14)

- ✓ Instalador unificado `Stockly-Setup.exe` (IExpress + winget) que instala Node LTS, JDK 17, Maven y MySQL si faltan, prepara `.env`, inicializa el datadir de MySQL y compila la CLI Java.
- ✓ Refactor de todos los scripts `.bat` para usar **rutas relativas** (detección dinámica de Node/MySQL/Java con `where` + fallbacks a `%ProgramFiles%`). Antes había rutas hardcodeadas a `C:\...\jdk-17.0.19.10-hotspot`, `D:\Program Files\nodejs`, etc.
- ✓ `start.bat` ahora detecta puertos ocupados: si hay un servicio MySQL del sistema en :3306, levanta nuestra instancia en :3307 y sincroniza `DB_PORT` y `DB_PASSWORD` (vacío) en `backend\.env`.
- ✓ Fix UX login (`app.js`): el helper `api()` cerraba sesión y mostraba "Sesión expirada" ante cualquier 401, incluso el del propio `/auth/login` cuando la contraseña era incorrecta. Ahora distingue endpoints de auth y solo dispara la lógica de expiración si había token activo.
- 🕒 Despliegue en VPS / Render con dominio propio.
- 🕒 Memoria + bitácora consolidada.
- 🕒 Vídeo demo de 3 minutos.

- 🕒 Defensa.

Sprint 6 — App móvil empleado (planificado, a partir del Día 15)

- 🕒 App Android nativa-ligera (Capacitor) que envuelve la web y expone vistas optimizadas para operarios:
 - Listado de **reservas asignadas** con su ubicación en almacén.
 - Acción de **confirmar entrega** (cambio de estado + firma/fotografía opcional).
 - Acción de **dar de alta una incidencia** (rotura, falta de stock, error de pedido).
- 🕒 Sesión persistente con biometría (`@capacitor/biometric-auth`) para no introducir contraseña en cada turno.
- 🕒 Notificaciones push (OneSignal o FCM) al asignar una nueva reserva.
- Detalle completo en `mobile.md` §6.

3. Incidencias notables

Día	Incidencia	Solución
Día 4	El frontend no leía la API en otro puerto	Configuración explícita de CORS + URL absoluta cuando el origin no es 3001
Día 5	Conflictos al cancelar reservas en paralelo	Bloqueo <code>FOR UPDATE</code> y comprobación de estado previo
Día 6	Token JWT no se enviaba en peticiones <code>fetch</code>	Helper <code>api()</code> centralizado que añade el header
Día 9	Reload del Service Worker no aplicaba CSS nuevo	Versionado de <code>CACHE</code> y <code>clients.claim()</code> en <code>activate</code>
Día 10	Móvil: el toast quedaba debajo de la bottom-nav	<code>bottom: calc(80px + safe-area-inset-bottom)</code>
Día 11	<code>start.bat</code> : MySQL no respondía en 30 s	Servicio MySQL del sistema bloqueaba :3306; conmutar a :3307 y sincronizar <code>DB_PORT</code> / <code>DB_PASSWORD</code> en <code>.env</code>
Día 12	"Failed to fetch" tras login	<code>DB_PASSWORD=123</code> heredada en <code>.env</code> rompía auth contra nuestra instancia passwordless; forzar valores correctos en cada arranque
Día 13	401 en login se mostraba como "Sesión expirada"	<code>api()</code> distingue ahora <code>/auth/login</code> y solo trata como expiración si había token previo
Día 14	Rutas absolutas en <code>gui.bat</code> , <code>stockly.bat</code> , <code>consola.bat</code> , <code>install.bat</code> , <code>reset_mysql.bat</code>	Sustituidas por detección dinámica de Java/Node/MySQL en PATH con fallback a <code>%ProgramFiles%</code>

4. Reparto diario (plantilla)

Plantilla a copiar al final del documento por sprint o jornada relevante.

Día N

- Adrián Bravo Santos: `_qué hizo_, _bloqueos_, _siguientes pasos_`.
- Miguel Ángel Florido: `_qué hizo_, _bloqueos_, _siguientes pasos_`.
- Hitos cerrados: `_PR #x, ticket #y_`.
- Pendiente: `_items para el día siguiente_`.
- Notas / aprendizajes: `_qué nos llevamos como equipo_`.

5. Reflexión técnica final (a completar al cierre)

- *Qué cambiaríamos del modelo de datos en una v2.*
- *Qué partes del código nos sentimos orgullosos de cómo quedaron.*
- *Qué dependencia o framework hubiera ahorrado tiempo.*
- *Qué métricas reales obtuvimos en producción (tamaño bundle, tiempos de respuesta).*

6. Material entregable (checklist final)

- ☐ Repositorio público con commits limpios.
- ☐ README + bitácora + roadmap.
- ☐ BD versionada en `db/schema.sql`.
- ☐ URL del despliegue accesible.
- ☐ Vídeo demo (≤ 3 min) subido a YouTube/no listado.
- ☐ Memoria PDF (~30-50 páginas).
- ☐ Presentación de defensa (≤ 15 slides).
- ☐ Diagramas (E/R, casos de uso, despliegue) en `docs/diagramas/`.